

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
Andromeda Alike Ramadhani
109082500156
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

1. Skema Pemrosesan Sekuensial

Pemrosesan sekuensial merupakan salah satu skema mendasar dalam logika pemrograman komputer, di mana sekumpulan data masukan diproses secara berurutan baris demi baris, mulai dari elemen pertama hingga elemen terakhir. Dalam implementasinya, efisiensi skema sekuensial ini sangat bergantung pada penggunaan struktur kontrol perulangan (looping) seperti for atau while, serta struktur percabangan (conditional). Secara umum, terdapat dua pola utama dalam pembacaan data sekuensial, yaitu pembacaan tanpa penanda dan pembacaan dengan menggunakan penanda berakhirnya data (marker / sentinel).

Pada pola pembacaan data tanpa marker, program biasanya sudah mengetahui jumlah mutlak dari data yang harus diproses sejak awal (misalnya didefinisikan oleh variabel N). Sebaliknya, pada skema pembacaan data dengan marker, jumlah data masukan bersifat dinamis atau tidak diketahui secara pasti. Program akan terus membaca input secara berulang sampai mendeteksi suatu nilai khusus yang disepakati sebagai sinyal pemberhentian (marker), contohnya angka 9999. Di dalam skema dengan marker ini, terdapat pula variasi penanganan khusus, seperti pengujian kasus kosong di mana data pertama langsung berupa marker hingga pemrosesan elemen pertama sebagai kasus khusus untuk inisialisasi variabel tertentu (seperti pada algoritma pencarian nilai maksimum atau minimum) agar terhindar dari bias nilai batas.

2. Metode Monte Carlo

Metode Monte Carlo adalah sebuah teknik analisis numerik dan simulasi stokastik yang memanfaatkan pembangkitan bilangan acak (random generator) secara berulang untuk menyelesaikan suatu permasalahan matematis atau memodelkan sistem fisik yang kompleks. Pendekatan ini bekerja berdasarkan prinsip statistik frekuensi relatif, di mana keakuratan hasil estimasi atau prediksi numerik sangat ditentukan oleh besarnya kuantitas sampel acak yang digunakan dalam pengujian. Berdasarkan hukum bilangan besar, semakin banyak volume data sampel acak yang dibangkitkan ke dalam sistem, maka deviasi atau tingkat kesalahan (error) komputasi akan semakin mengecil dan nilainya akan bergerak konvergen menuju nilai matematis yang sebenarnya.

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

Statement Skema Pemrosesan Sekuensial

1. Soal1

Source Code:

```
package main

import "fmt"

func main() {
    var bilangan, total, rata2 float64
    var hitung int

    total = 0
    hitung = 0

    for {
        fmt.Scan(&bilangan)
        if bilangan == 9999 {
            break
        }
        total += bilangan
        hitung++
    }

    if hitung > 0 {
        rata2 = total / float64(hitung)
        fmt.Printf("Jumlah bilangan yang valid: %d\n",
hitung)
        fmt.Printf("Total penjumlahan: %.2f\n", total)
        fmt.Printf("Rata-Rata: %.2f\n", rata2)
    } else {
        fmt.Println("Tidak ada bilangan valid!")
    }
}
```

```
}  
  
}
```

The screenshot shows a Go IDE with a source code editor on the left and an output window on the right. The source code is a Go program that reads integers from standard input until the user enters 9999. It then calculates the sum of the valid numbers and the average. The output window displays the results of the program's execution.

```
Soal1 > go no1.go > main  
1 package main  
2  
3 import "fmt"  
4  
5 func main() {  
6     var bilangan, total, rata2 float64  
7     var hitung int  
8  
9     total = 0  
10    hitung = 0  
11  
12    for {  
13        fmt.Scan(&bilangan)  
14        if bilangan == 9999 {  
15            break  
16        }  
17        total += bilangan  
18        hitung++  
19    }  
20  
21    if hitung > 0 {  
22        rata2 = total / float64(hitung)  
23        fmt.Printf("Jumlah bilangan yang valid: %d\n", hitung)  
24        fmt.Printf("Total penjumlahan: %.2f\n", total)  
25        fmt.Printf("Rata-Rata: %.2f\n", rata2)  
26    } else {  
27        fmt.Println("Tidak ada bilangan valid!")  
28    }  
29 }  
30
```

Output:

```
109082500156  
IF-13-02  
Andromeda Alika Ramadhani  
Ln 3, Col 26 | 47 characters | AA Forma... | 100% | Windows (CI) | UTF-8
```

Output:

The screenshot shows a terminal window where the Go program is being executed. The user enters several numbers, and the program outputs the sum and average of the valid numbers.

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\L  
ATSOL MODUL17\Soal1\nol.go"  
12.5  
5.0  
10.5  
2.0  
9999  
Jumlah bilangan yang valid: 4  
Total penjumlahan: 30.00  
Rata-Rata: 7.50  
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17>
```

Deskripsi Program:

Program diatas digunakan untuk menghitung total dan rata-rata dari sejumlah bilangan real yang diinputkan pengguna. Pertama, program dimulai dengan mendeklarasikan beberapa variabel yang diperlukan, yaitu bilangan untuk menyimpan input terkini, total untuk menghitung jumlah seluruh bilangan, rata2 untuk hasil pembagian akhir, serta hitung sebagai pencatat banyaknya data yang telah dimasukkan. Pada kondisi awal, variabel total dan hitung diinisialisasi dengan 0. Selanjutnya, kita gunakan perulangan tak terbatas untuk membaca input dari pengguna secara terus-menerus menggunakan `fmt.Scan`. Di dalam perulangan terdapat percabangan `if`. Apabila pengguna menginputkan angka 9999, nilai tersebut akan dikenali sebagai tanda berhenti dan program akan menjalankan perintah `break` untuk keluar dari perulangan. Jika angka yang diinputkan bukan 999, nilai tersebut akan dianggap valid dan akan ditambahkan ke dalam variabel total dan variabel hitung akan bertambah 1.

Setelah itu, program akan menjalankan validasi data terlebih dahulu melalui `if hitung > 0` untuk memastikan setidaknya ada 1 bilangan valid yang diinputkan pengguna sebelum operasi pembagian dilakukan. Jika kondisi tersebut terpenuhi, program akan menghitung rata-rata dengan membagi variabel total dengan nilai hitung yang sudah dikonversi menjadi tipe data `float64` agar sesuai dengan tipe data operannya. Output akhirnya adalah jumlah bilangan valid, total penjumlahan, dan nilai rata-ratanya. Namun, jika pengguna langsung menginputkan angka 9999 di awal program maka nilai hitung akan tetap 0 dan program akan menjalankan blok `else` dan memberikan output "Tidak ada bilangan valid!"

2. Soal2

Source Code:

```
package main

import "fmt"

func analisisString(x string, n int) (bool, int, int,
bool) {
    var ketemu, minimalDua bool
    var posisiPertama, jumlahX, i int
    var currentString string

    ketemu = false
    minimalDua = false
    posisiPertama = -1
    jumlahX = 0

    fmt.Printf("Masukkan %d data string:\n", n)

    for i = 1; i <= n; i++ {
        fmt.Printf("Data ke-%d: ", i)
        fmt.Scan(&currentString)

        if currentString == x {
            ketemu = true
```

```

        jumlahX = jumlahX + 1

        if posisiPertama == -1 {
            posisiPertama = i
        }
    }

    if jumlahX >= 2 {
        minimalDua = true
    }

    return ketemu, posisiPertama, jumlahX, minimalDua
}

func main() {
    var x string
    var n, posisi, totalX int
    var adaX, minimalDua bool

    fmt.Print("Masukkan X: ")
    fmt.Scan(&x)
    fmt.Print("Masukkan N: ")
    fmt.Scan(&n)

    adaX, posisi, totalX, minimalDua =
    analisisString(x, n)

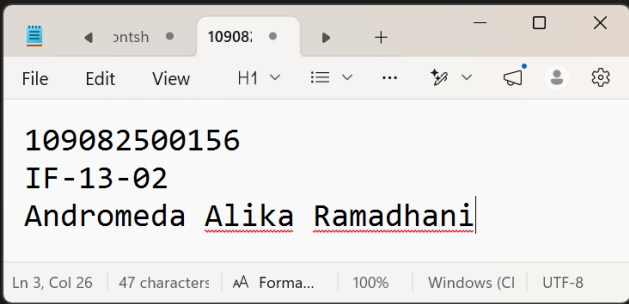
    fmt.Println("HASIL ANALISIS:")
    if adaX == true {
        fmt.Println("a. Apakah string x ada dalam
        kumpulan n data string tersebut? Ya, ada")
    } else {

```

```
        fmt.Println("a. Apakah string x ada dalam  
kumpulan n data string tersebut? Tidak ada")  
    }  
  
    if posisi != 1 {  
        fmt.Printf("b. Pada posisi ke berapa string x  
tersebut ditemukan? Posisi ke-%d\n", posisi)  
    } else {  
        fmt.Println("b. Pada posisi ke berapa string x  
tersebut ditemukan? Tidak ditemukan")  
    }  
  
    fmt.Printf("c. Ada berapakah string x dalam  
kumpulan n data string tersebut? Ada %d\n", totalX)  
  
    if minimalDua == true {  
        fmt.Println("d. Adakah sedikitnya dua string x  
dalam n data string tersebut? Ya, ada sedikitnya dua")  
    } else {  
        fmt.Println("d. Adakah sedikitnya dua string x  
dalam n data string tersebut? Tidak, kurang dari dua")  
    }  
}
```



```
Soal2 > no2.go > analisisString
1 package main
2
3 import "fmt"
4
5 func analisisString(x string, n int) (bool, int, int, bool) {
6     var ketemu, minimalDua bool
7     var posisiPertama, jumlahX, i int
8     var currentString string
9
10    ketemu = false
11    minimalDua = false
12    posisiPertama = -1
13    jumlahX = 0
14
15    fmt.Printf("Masukkan %d data string:\n", n)
16
17    for i = 1; i <= n; i++ {
18        fmt.Printf("Data ke-%d: ", i)
19        fmt.Scan(&currentString)
20
21        if currentString == x {
22            ketemu = true
23            jumlahX = jumlahX + 1
24
25            if posisiPertama == -1 {
26                posisiPertama = i
27            }
28        }
29    }
30}
```



Output:

```
ATSOL MODUL17\Soal2\n02.go"
Masukkan X: golang
Masukkan N: 5
Masukkan 5 data string:
Data ke-1: java
Data ke-2: golang
Data ke-3: python
Data ke-4: golang
Data ke-5: c++
HASIL ANALISIS:
a. Apakah string x ada dalam kumpulan n data string tersebut? Ya, ada
b. Pada posisi ke berapa string x tersebut ditemukan? Posisi ke-2
c. Ada berapakah string x dalam kumpulan n data string tersebut? Ada 2
d. Adakah sedikitnya dua string x dalam n data string tersebut? Ya, ada sedikitnya dua
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17>
```

Deskripsi Program:

Program diatas digunakan untuk menganalisis keberadaan sebuah string di dalam sekumpulan data string yang diinputkan oleh pengguna. Pertama, fungsi analisisString yang bertugas untuk menerima parameter x sebagai string yang dicari dan n sebagai jumlah data, serta mengembalikan empat buah nilai sekaligus berupa dua Boolean dan dua integer. Di dalam fungsi ini, kita inisiasikan variabel ketemu dan minimal dua dengan nilai false, variabel posisiPertama dengan nilai -1 sebagai penanda data belum ditemukan, serta variabel jumlahX dengan nilai 0. Proses pencarian dilakukan dengan perulangan for yang dijalankan sebanyak n kali untuk meminta input string secara berurutan. Setiap kali input dibaca, program akan melakukan pengecekan menggunakan percabangan if. Jika input sama dengan nilai x maka status ketemu berubah menjadi true dan jumlahX akan bertambah 1. Selain itu, terdapat kondisi nested if untuk mencatat indeks pertama kali x ditemukan, di mana nilai posisiPertama hanya akan diisi oleh indeks i saat variabel tersebut masih bernilai -1. Setelah perulangan selesai, program akan melakukan evaluasi akhir untuk mengubah status minimalDua

menjadi true. Jika jumlahX mencapai angka 2 atau lebih, seluruh hasil evaluasi tersebut dikembalikan ke fungsi pemanggil.

Selanjutnya fungsi main diawali dengan mendeklarasikan variabel yang diperlukan untuk menampung input pengguna serta hasil pengembalian dari fungsi analisisString. Pertama, program akan meminta input string target x dan jumlah perulangan n. Setelah itu, fungsi analisisString dipanggil lalu hasil kembaliannya ditampung secara berurutan ke dalam variabel adaX, posisi, totalX, dan minimalDua. Terakhir adalah program akan menampilkan output menggunakan percabangan if-else dimana program akan mencetak status keberadaan string x, posisi pertama kali ditemukan, jumlah total kemunculan x, serta kepastian apakah x muncul minimal dua kali atau kurang dari itu berdasarkan hasil pemrosesan data yang telah dilakukan sebelumnya.

3. Soal3

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func hitungTetesanHujan(totalTetesan int) (int, int,
int, int) {
    var x, y float64
    var countA int = 0
    var countB int = 0
    var countC int = 0
    var countD int = 0
    var i int

    for i = 1; i <= totalTetesan; i++ {
        x = rand.Float64()
        y = rand.Float64()
```

```
        if x < 0.5 && y < 0.5 {
            countA = countA + 1
        } else if x >= 0.5 && y < 0.5 {
            countB = countB + 1
        } else if x >= 0.5 && y >= 0.5 {
            countC = countC + 1
        } else if x < 0.5 && y >= 0.5 {
            countD = countD + 1
        }
    }

    return countA, countB, countC, countD
}

func main() {
    rand.Seed(time.Now().UnixNano())

    var input int
    fmt.Print("Masukkan banyaknya tetesan air hujan: ")
    fmt.Scan(&input)

    var tetesanA int = 0
    var tetesanB int = 0
    var tetesanC int = 0
    var tetesanD int = 0

    tetesanA, tetesanB, tetesanC, tetesanD =
hitungTetesanHujan(input)

    var faktorKonversi float64 = 0.0001
```

```

    var curahHujanA float64 = float64(tetesanA) *
    faktorKonversi

    var curahHujanB float64 = float64(tetesanB) *
    faktorKonversi

    var curahHujanC float64 = float64(tetesanC) *
    faktorKonversi

    var curahHujanD float64 = float64(tetesanD) *
    faktorKonversi

    fmt.Printf("Curah hujan daerah A: %.4f
    milimeter\n", curahHujanA)

    fmt.Printf("Curah hujan daerah B: %.4f
    milimeter\n", curahHujanB)

    fmt.Printf("Curah hujan daerah C: %.4f
    milimeter\n", curahHujanC)

    fmt.Printf("Curah hujan daerah D: %.4f
    milimeter\n", curahHujanD)

}

```

```

Soal3 > -go no3.go > main
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "time"
7 )
8
9 func hitungTetesanHujan(totalTetesan int) (int,
10     var x, y float64
11     var countA int = 0
12     var countB int = 0
13     var countC int = 0
14     var countD int = 0
15     var i int
16
17     for i = 1; i <= totalTetesan; i++ {
18         x = rand.Float64()
19         y = rand.Float64()
20
21         if x < 0.5 && y < 0.5 {
22             countA = countA + 1
23         } else if x >= 0.5 && y < 0.5 {
24             countB = countB + 1
25         } else if x >= 0.5 && y >= 0.5 {
26             countC = countC + 1
27         } else if x < 0.5 && y >= 0.5 {
28             countD = countD + 1
29         }
30     }

```

109082500156
IF-13-02
Andromeda Alika Ramadhani

Ln 3, Col 26 | 47 characters | AA Forma... | 100% | Windows (CI | UTF-8

Ln 58, Col 69 | Tab Size: 4 | UTF-8 | CRLF | Go | 1.25.1 | Go Live

Output:

```

PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal3\n03.go"
Masukkan banyaknya tetesan air hujan: 10000000
Curah hujan daerah A: 249.8608 milimeter
Curah hujan daerah B: 249.7860 milimeter
Curah hujan daerah C: 250.1939 milimeter
Curah hujan daerah D: 250.1593 milimeter
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> 

```

Deskripsi Program:

Program diatas digunakan untuk mengukur curah hujan dari 4 daerah yang berdekatan. Fungsi `hitungTetesanHujan` digunakan untuk menerima satu parameter input berupa `totalTetesan` dengan nilai integer dan mengembalikan 4 nilai keluaran yang merepresentasikan jumlah tetesan di masing-masing daerah. Di dalam fungsi ini, kita buat variabel penghitung `countA`, `countB`, `countC`, `countD` yang diinisialisasi dengan nilai 0 di awal. Selanjutnya, kita jalankan fungsi `for` sebanyak jumlah tetesan yang diminta. Pada setiap iterasi, kita panggil fungsi `rand.Float64()` sebanyak 2 kali untuk menghasilkan koordinat acak `x` dan `y` dalam rentang `[0.0,1.0]`. Nilai koordinat tersebut di evaluasi menggunakan `if-else if` untuk memetakan posisi tetesan ke dalam 4 daerah. Jika koordinat memenuhi `x < 0.5` dan `y < 0.5` masuk ke daerah A, kondisi `x >= 0.5` dan `y < 0.5` masuk ke daerah B, kondisi `x >= 0.5` dan `y >= 0.5` masuk ke daerah C, kondisi `x < 0.5` dan `y >= 0.5` masuk ke daerah D. Jika seluruh iterasi sudah selesai maka akumulasi tetesan dari tiap daerah dikembalikan ke fungsi pemanggil.

Fungsi `main` digunakan untuk menjalankan program. Eksekusi diawali dengan pemanggilan `rand.Seed(time.Now().UnixNano())` yang berfungsi untuk melakukan seeding pada generator bilangan acak berbasis waktu sistem agar angka yang dihasilkan selalu bervariasi setiap kali program dijalankan. Selanjutnya program meminta input pengguna untuk menentukan banyaknya tetesan air hujan yang akan dihitung melalui `fmt.Scan`. Nilai tersebut dikirim sebagai argumen saat memanggil fungsi `hitungTetesanHujan` dan hasilnya ditampung ke dalam 4 variabel yaitu `tetesanA`, `tetesanB`, `tetesanC`, `tetesanD`. Selanjutnya jumlah tetesan dikonversi menjadi nilai curah hujan dalam satuan milimeter dengan mendeklarasikan variabel `faktorKonversi` bernilai `0.0001`. Nilai curah hujan di setiap daerah dihitung dengan mengalikan jumlah tetesan yang telah dikonversi ke tipe data `float64` dengan faktor konversi tersebut. Langkah terakhir adalah menampilkan hasil perhitungan curah hujan untuk daerah A, B, C, dan D ke layar dengan format presisi empat angka di belakang koma (`%.4f`).

4. Soal4

Source Code:

```

package main

import "fmt"

func hitungSuku(i int) float64 {
    var pembilang float64
    var penyebut float64

```

```

        if i%2 != 0 {
            pembilang = 1.0
        } else {
            pembilang = -1.0
        }

        penyebut = float64(2*i - 1)

        return pembilang / penyebut
    }

func hitungPiDeret(n int) {
    var totalDeret float64 = 0.0
    var hasilPi float64
    var i int

    for i = 1; i <= n; i++ {
        totalDeret = totalDeret + hitungSuku(i)
    }

    hasilPi = 4.0 * totalDeret

    fmt.Printf("Hasil Pi: %.7f\n", hasilPi)
}

func main() {
    var n int

    fmt.Print("N suku pertama: ")
    fmt.Scan(&n)

```

```
hitungPiDeret(n)
```

```
}
```

```
Soal4 > no4.go > hitungSuku
1 package main
2
3 import "fmt"
4
5 func hitungSuku(i int) float64 {
6     var pembilang float64
7     var penyebut float64
8
9     if i%2 != 0 {
10         pembilang = 1.0
11     } else {
12         pembilang = -1.0
13     }
14
15     penyebut = float64(2*i - 1)
16
17     return pembilang / penyebut
18 }
19
20 func hitungPiDeret(n int) {
21     var totalDeret float64 = 0.0
22     var hasilPi float64
23     var i int
24
25     for i = 1; i <= n; i++ {
26         totalDeret = totalDeret + hitungSuku(i)
27     }
28
29     hasilPi = 4.0 * totalDeret
30 }
```

```
10908.2500156
IF-13-02
Andromeda Alika Ramadhani
```

Ln 3, Col 26 | 47 characters | AA Forma... | 100% | Windows (CI) | UTF-8

Output:

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal4\n04.go"
N suku pertama: 1000000
Hasil Pi: 3.1415917
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17>
```

Deskripsi Program:

Program diatas digunakan untuk menghitung S untuk 1000000 suku pertama berdasarkan rumus hampiran deret Leibniz. Fungsi hitungSuku digunakan untuk menerima satu parameter i dengan nilai integer yang merepresentasikan urutan indeks suku deret dan mengembalikan sebuah nilai desimal. Di dalam fungsi ini, program mengevaluasi nilai pembilang berdasarkan kondisi ganjil atau genap dari indeks i menggunakan operasi modulus ($i \% 2 \neq 0$). Jika indeks bernilai ganjil, variabel pembilang diisi nilai 1.0, sedangkan jika genap akan diisi nilai -1.0 untuk menghasilkan pola tanda yang berselang-seling (+ dan -). Nilai variabel penyebut dihitung dengan rumus $2i - 1$ yang kemudian dikonversi ke tipe data float64. Fungsi ini diakhiri dengan mengembalikan hasil pembagian antara variabel pembilang dan penyebut.

Fungsi hitungPiDeret digunakan untuk mengakumulasi nilai deret dan menampilkan hasil akhirnya. Fungsi ini menerima parameter n yang menentukan batas jumlah perulangan. Di dalamnya, deklarasi variabel totalDeret diinisialisasi dengan nilai awal 0.0. Melalui struktur perulangan for yang berjalan dari angka 1 hingga n, program berulang kali memanggil fungsi hitungSuku(i) untuk menjumlahkan setiap suku deret ke dalam variabel totalDeret. Setelah proses perulangan selesai, akumulasi deret tersebut dikalikan dengan konstanta 4.0 sesuai dengan formula Leibniz untuk

mendapatkan nilai akhir hampiran Pi yang kemudian ditampung dalam variabel hasilPi. Hasil perhitungan tersebut dicetak ke layar menggunakan format presisi tujuh angka di belakang koma (%.7f). Fungsi main digunakan untuk mengeksekusi keseluruhan alur program. Fungsi ini bertugas untuk mendeklarasikan variabel input n, meminta masukkan jumlah suku dari pengguna melalui perintah fmt.Scan, dan meneruskan nilai tersebut saat memanggil fungsi hitungPiDeret.

5. Soal4 Modifikasi

Source Code:

```
package main

import (
    "fmt"
    "math"
)

func hitungSuku(i int) float64 {
    var pembilang float64
    var penyebut float64

    if i%2 != 0 {
        pembilang = 1.0
    } else {
        pembilang = -1.0
    }

    penyebut = float64(2*i - 1)

    return pembilang / penyebut
}

func simulasiKonvergensi() {
    var totalDeret float64 = 0.0
```



```

        var selisih, sukuSekarang, sukuBerikutnya,
piSekarang, piBerikutnya float64

        var i int = 1

        for {

            sukuSekarang = hitungSuku(i)
            sukuBerikutnya = hitungSuku(i + 1)

            totalDeret = totalDeret + sukuSekarang

            selisih = math.Abs(sukuBerikutnya -
sukuSekarang)

            if selisih <= 0.00001 && i > 1 {
                break
            }
            i += 1
        }

        fmt.Printf("Hasil PI: %.10f\n", piSekarang)
        fmt.Printf("Hasil PI: %.10f\n", piBerikutnya)
        fmt.Printf("Pada i ke: %d\n", i)
    }

func main() {
    var n int

    fmt.Print("N suku pertama: ")
    fmt.Scan(&n)

    simulasiKonvergenSI()
}

```

The image shows a Go program in a code editor and its output in a terminal window. The code defines a function `hitungSuku` that calculates the Leibniz series term for a given index `i`. It uses `float64` for precision. The `simulasiKonvergensi` function iterates through terms, accumulating the sum in `totalDeret` until the difference between consecutive terms is less than 0.0001.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func hitungSuku(i int) float64 {
9     var pembilang float64
10    var penyebut float64
11
12    if i%2 != 0 {
13        pembilang = 1.0
14    } else {
15        pembilang = -1.0
16    }
17
18    penyebut = float64(2*i - 1)
19
20    return pembilang / penyebut
21 }
22
23 func simulasiKonvergensi() {
24     var totalDeret float64 = 0.0
25     var selisih, sukuSekarang, sukuBerikutnya, piSekarang, piBerikutnya float64
26     var i int = 1
27
28     for {
29         sukuSekarang = hitungSuku(i)
30         sukuBerikutnya = hitungSuku(i+1)
31         selisih = math.Abs(sukuSekarang - sukuBerikutnya)
32         totalDeret = totalDeret + sukuSekarang
33         piSekarang = totalDeret
34         piBerikutnya = totalDeret + sukuBerikutnya
35
36         if selisih < 0.0001 {
37             break
38         }
39         i++
40     }
41
42     fmt.Println("N suku pertama: 100000")
43     fmt.Println("Hasil PI: 0.0000000000")
44     fmt.Println("Hasil PI: 0.0000000000")
45     fmt.Println("Pada i ke: 100001")
46 }
```

The terminal output shows the program's execution results:

```
109082500156
IF-13-02
Andromeda Alika Ramadhani
```

Output:

The terminal shows the output of the program, including the number of terms, the calculated value of Pi, and the index where the convergence condition was met.

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal4rev\n04rev.go"
N suku pertama: 100000
Hasil PI: 0.0000000000
Hasil PI: 0.0000000000
Pada i ke: 100001
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17>
```

Deskripsi Program:

Program modifikasi ini digunakan untuk mensimulasikan kondisi konvergensi pada hampiran nilai Pi menggunakan rumus Leibniz berdasarkan selisih nilai antara dua suku yang berurutan. Fungsi `hitungSuku` digunakan untuk menerima satu parameter `i` dengan nilai integer yang merepresentasikan urutan indeks suku deret dan mengembalikan sebuah nilai desimal. Di dalam fungsi ini, program mengevaluasi nilai pembilang berdasarkan kondisi ganjil atau genap dari indeks `i` menggunakan operasi modulus ($i \% 2 \neq 0$). Jika indeks bernilai ganjil, variabel pembilang diisi nilai 1.0, sedangkan jika genap akan diisi nilai -1.0 untuk menghasilkan pola tanda yang berselang-seling (+ dan -). Nilai variabel penyebut dihitung dengan rumus $2i - 1$ yang kemudian dikonversi ke tipe data `float64`. Fungsi ini diakhiri dengan mengembalikan hasil pembagian antara variabel pembilang dan penyebut.

Fungsi `simulasiKonvergensi` digunakan untuk melakukan proses iterasi pencarian batas konvergensi menggunakan struktur perulangan tak terbatas. Di dalam perulangan ini, program menghitung nilai `sukuSekarang` pada indeks `i` dan `sukuBerikutnya` pada indeks `i + 1` melalui pemanggilan fungsi `hitungSuku`. Variabel `totalDeret` terus diakumulasi dengan menambahkan nilai dari `sukuSekarang` pada setiap iterasinya. Batas pemberhentian perulangan dievaluasi berdasarkan nilai absolut dari selisih antara `sukuBerikutnya` dan `sukuSekarang` menggunakan fungsi `math.Abs`. Jika nilai selisih tersebut sudah lebih kecil atau sama dengan tingkat toleransi 0.0001, maka program mengeksekusi perintah `break` untuk keluar dari perulangan. Setelah

keluar dari looping, program mencetak nilai dari variabel piSekarang, piBerikutnya, serta indeks posisi i terakhir ke layar. Pada akhir program, fungsi main bertindak sebagai entry point yang mendeklarasikan variabel input n dan meminta masukkan dari pengguna, Terakhir, kita panggil fungsi simulasiKonvergensi untuk mencetak hasilnya ke layar.

6. Soal5

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func hitungToppingPizza(totalTopping int) int {
    var i int
    var countPizza int = 0
    var x, y, jarak float64

    for i = 1; i <= totalTopping; i++ {
        x = rand.Float64()
        y = rand.Float64()

        jarak = (x-0.5)*(x-0.5) + (y-0.5)*(y-0.5)

        if jarak <= 0.25 {
            countPizza = countPizza + 1
        }
    }

    return countPizza
}
```

```

func main() {
    rand.Seed(time.Now().UnixNano())

    var inputTopping, toppingPizza int

    fmt.Print("Banyak Topping: ")
    fmt.Scan(&inputTopping)

    toppingPizza = hitungToppingPizza(inputTopping)

    fmt.Printf("Topping pada pizza: %d\n",
toppingPizza)
}

```

The screenshot shows a Go IDE with a code editor on the left and a terminal/output window on the right. The code in the editor defines a function `hitungToppingPizza` that calculates the number of pizzas based on the total number of toppings. The `main` function calls this function and prints the result. The output window shows the execution results: a random seed, the number of pizzas (13), and the name of the person (Andromeda Alike Ramadhani).

```

Soal5 > no5.go > hitungToppingPizza
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "time"
7 )
8
9 func hitungToppingPizza(totalTopping int) int {
10     var i int
11     var countPizza int = 0
12     var x, y, jarak float64
13
14     for i = 1; i <= totalTopping; i++ {
15         x = rand.Float64()
16         y = rand.Float64()
17
18         jarak = (x-0.5)*(x-0.5) + (y-0.5)*(y-0.5)
19
20         if jarak <= 0.25 {
21             countPizza = countPizza + 1
22         }
23     }
24     return countPizza
25 }
26
27 func main() {
28     rand.Seed(time.Now().UnixNano())
29

```

Output:

```

109082500156
IF-13-02
Andromeda Alike Ramadhani

```

Ln 3, Col 26 | 47 characters | AA Forma... | 100% | Windows (CI) | UTF-8

Ln 10, Col 14 | Tab Size: 4 | UTF-8 | CRLF | Go | 1.25.1 | Go Live

Output:

```

PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal5\no5.go"
Banyak Topping: 1234567
Topping pada pizza: 969336
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal5\no5.go"
Banyak Topping: 1000000
Topping pada pizza: 7853234
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> 

```

Deskripsi Program:

Program diatas digunakan untuk mensimulasikan penempatan posisi topping pada sebuah adonan pizza berbentuk lingkaran dengan memanfaatkan metode Monte Carlo. Fungsi `hitungToppingPizza` digunakan untuk menerima satu parameter input `totalTopping` bernilai integer yang menentukan jumlah iterasi, serta mengembalikan sebuah nilai integer yang merepresentasikan jumlah topping yang berhasil masuk ke dalam area permukaan pizza. Di dalam fungsi ini, variabel penghitung `countPizza` diinisialisasi dengan nilai awal 0. Proses penentuan posisi dilakukan di dalam perulangan `for` dengan membangkitkan koordinat acak `x` dan `y` dalam rentang interval `[0.0, 1.0]` menggunakan fungsi `rand.Float64()`. Nilai koordinat tersebut kemudian digunakan untuk menghitung kuadrat jarak titik dari pusat lingkaran (asumsi titik pusat berada pada koordinat `(0.5, 0.5)`) melalui persamaan $\text{jarak} = (x-0.5)*(x-0.5) + (y-0.5)*(y-0.5)$. Menggunakan kondisi percabangan `if`, apabila nilai jarak kurang dari atau sama dengan kuadrat jari-jari lingkaran, yaitu 0.25 (dari jari-jari `r = 0.5`), maka titik acak tersebut dinyatakan berada di dalam area pizza dan variabel `countPizza` akan bertambah 1. Setelah seluruh perulangan selesai dieksekusi, fungsi ini akan mengembalikan akumulasi nilai `countPizza`.

Sementara itu, fungsi `main` bertindak sebagai instruksi utama untuk mengendalikan alur jalannya simulasi program. Eksekusi program diawali dengan pemanggilan fungsi `rand.Seed(time.Now().UnixNano())` yang bertujuan untuk melakukan seeding pada generator bilangan acak berbasis waktu Unix dalam satuan nanosekon, sehingga koordinat titik acak yang dihasilkan selalu bervariasi setiap kali program dijalankan. Program kemudian meminta input jumlah total topping dari pengguna melalui perintah `fmt.Scan` untuk disimpan ke dalam variabel `inputTopping`. Nilai input tersebut selanjutnya dilewatkan sebagai parameter saat memanggil fungsi `hitungToppingPizza`, di mana hasil kembalian dari fungsi tersebut akan ditampung oleh variabel `toppingPizza`. Langkah terakhir dari program ini adalah menampilkan jumlah topping yang valid atau berhasil mendarat di dalam area pizza ke layar menggunakan fungsi `fmt.Printf`.

7. Soal5 Modifikasi

Source Code:

```

package main

import (
    "fmt"
    "math/rand"

```

```

        "time"
    )

    func hitungToppingPizza(totalTopping int) int {
        var i int
        var countPizza int = 0
        var x, y, jarak float64

        for i = 1; i <= totalTopping; i++ {
            x = rand.Float64()
            y = rand.Float64()

            jarak = (x-0.5)*(x-0.5) + (y-0.5)*(y-0.5)

            if jarak <= 0.25 {
                countPizza = countPizza + 1
            }
        }
        return countPizza
    }

    func main() {
        rand.Seed(time.Now().UnixNano())

        var inputTopping, toppingPizza int
        var nilaiPi float64

        fmt.Print("Banyak Topping: ")
        fmt.Scan(&inputTopping)

        toppingPizza = hitungToppingPizza(inputTopping)
    }

```

```

        nilaiPi = 4.0 * float64(toppingPizza) /
float64(inputTopping)

        fmt.Printf("Topping pada pizza: %d\n",
toppingPizza)

        fmt.Printf("PI: %.10f\n", nilaiPi)

    }

```

The screenshot shows a Go IDE with the source code for 'no5rev.go' on the left and a browser window on the right displaying the program's output.

Source Code (no5rev.go):

```

1  package main
2
3  import (
4      "fmt"
5      "math/rand"
6      "time"
7  )
8
9  func hitungToppingPizza(totalTopping int) int {
10     var i int
11     var countPizza int = 0
12     var x, y, jarak float64
13
14     for i = 1; i <= totalTopping; i++ {
15         x = rand.Float64()
16         y = rand.Float64()
17
18         jarak = (x-0.5)*(x-0.5) + (y-0.5)*(y-0.5)
19
20         if jarak <= 0.25 {
21             countPizza = countPizza + 1
22         }
23     }
24     return countPizza
25 }
26
27 func main() {
28     rand.Seed(time.Now().UnixNano())
29
30     banyakTopping := 1234567
31     fmt.Println("Banyak Topping:", banyakTopping)
32     toppingPizza := hitungToppingPizza(banyakTopping)
33     fmt.Println("Topping pada pizza:", toppingPizza)
34     pi := 4.0 * float64(toppingPizza) / float64(banyakTopping)
35     fmt.Println("PI:", pi)
36 }

```

Browser Output:

```

109082500156
IF-13-02
Andromeda Alike Ramadhani

```

Output:

The screenshot shows a PowerShell terminal with the following output:

```

PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal5rev\no5rev.go"
Banyak Topping: 1234567
Topping pada pizza: 969740
PI: 3.1419598936
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal5rev\no5rev.go"
Banyak Topping: 10
Topping pada pizza: 9
PI: 3.6000000000
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal5rev\no5rev.go"
Banyak Topping: 256
Topping pada pizza: 204
PI: 3.1875000000
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17\Soal5rev\no5rev.go"
Banyak Topping: 5000
Topping pada pizza: 3917
PI: 3.1336000000
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL17>

```

Deskripsi Program:

Program modifikasi diatas digunakan untuk menghitung nilai Pi berbasis simulasi Monte Carlo dengan menghitung rasio sebaran topping yang jatuh di dalam area adonan pizza. Fungsi `hitungToppingPizza` digunakan untuk menerima satu parameter input `totalTopping` bernilai integer yang menentukan jumlah iterasi, serta mengembalikan sebuah nilai integer yang merepresentasikan jumlah topping yang berhasil masuk ke dalam area permukaan pizza. Di dalam fungsi ini, variabel penghitung `countPizza` diinisialisasi dengan nilai awal 0. Proses penentuan posisi dilakukan di dalam perulangan `for` dengan membangkitkan koordinat acak `x` dan `y` dalam rentang interval `[0.0, 1.0]` menggunakan fungsi `rand.Float64()`. Nilai koordinat tersebut kemudian digunakan untuk menghitung kuadrat jarak titik dari pusat lingkaran (asumsi titik pusat berada pada koordinat `(0.5, 0.5)`) melalui persamaan jarak = $(x-0.5)*(x-0.5) + (y-0.5)*(y-0.5)$. Menggunakan kondisi percabangan `if`, apabila nilai jarak kurang dari atau sama dengan kuadrat jari-jari lingkaran, yaitu 0.25 (dari jari-jari $r = 0.5$), maka titik acak tersebut dinyatakan berada di dalam area pizza dan variabel `countPizza` akan bertambah 1. Setelah seluruh perulangan selesai dieksekusi, fungsi ini akan mengembalikan akumulasi nilai `countPizza`.

Sementara itu, fungsi `main` berfungsi sebagai pengendali utama dari seluruh jalannya program. Eksekusi diawali dengan perintah `rand.Seed(time.Now().UnixNano())` untuk melakukan seeding pada fungsi acak berbasis waktu Unix agar koordinat yang dihasilkan selalu unik di setiap pengujian. Program kemudian meminta pengguna memasukkan jumlah total topping yang diinginkan melalui perintah `fmt.Scan` untuk disimpan ke dalam variabel `inputTopping`. Nilai tersebut kemudian dilewatkan saat memanggil fungsi `hitungToppingPizza`, dan hasil kembaliannya ditampung oleh variabel `toppingPizza`. Untuk mengestimasi nilai Pi, program membagi variabel `toppingPizza` dengan `inputTopping` yang terlebih dahulu dikonversi ke tipe data `float64`, kemudian dikalikan dengan konstanta 4.0 (representasi rasio luas lingkaran terhadap luas persegi luar) dan hasilnya disimpan ke dalam variabel `nilaiPi`. Langkah terakhir dari program ini adalah menampilkan jumlah topping yang berada di dalam pizza serta hasil estimasi nilai Pi ke layar dengan format presisi sepuluh angka di belakang koma (`%.10f`).

DAFTAR PUSTAKA

Program Studi S1 Informatika. (2026). *Modul algoritma dan pemrograman 2 IF REG 2026*. Universitas Telkom.

Danau Data. (2020, March 14). *3.14.2020 estimasi nilai π dengan simulasi Monte Carlo pada cluster Hadoop*. Medium. <https://medium.com/danau-data/3-14-2020-estimasi-nilai-%CF%80-dengan-simulasi-monte-carlo-pada-cluster-hadoop-6cf035890be4>